Rapport final de qualification

Validation, robustesse et fiabilité d'un modèle de détection de fraude bancaire

Ndeye Khadidiatou LO

Projet : Test logiciel et assurance qualité d'un modèle IA (AI-TestGuard)
Jeu de données : Credit Card Fraud Detection (Kaggle, ULB)
Dépôt : github.com/Umm-abdallah/Test-logiciel-et-assurance-qualite-modele-IA

Synthèse

Ma conclusion tient en une phrase : le pipeline réussit les tests techniques, mais le modèle n'est pas autorisé à agir sur de vrais paiements. Je recommande seulement un pilote en mode observation, après correction d'une anomalie bloquante.

Côté technique, les 136 tests automatisés passent et couvrent 97 % du code. Je n'ai trouvé aucune fuite de données, les résultats se reproduisent à l'identique et un appel de scoring répond en 13,4 ms au 95e percentile.

Côté modèle, le bilan est plus nuancé. Il bloque très peu de clients honnêtes (12 sur 56 651, soit 0,021 %) et 85 % de ses alertes sont de vraies fraudes. Mais il laisse passer 27 % des fraudes : son rappel de 0,726 reste sous les 0,75 que je m'étais fixé. En creusant, j'ai aussi découvert qu'il ignore le montant et qu'il s'appuie en partie sur une variable temporelle qui n'a aucun sens en dehors du jeu Kaggle.

1. Contexte et périmètre de l'évaluation

Une banque veut s'appuyer sur un modèle d'apprentissage automatique pour repérer les paiements frauduleux. Ce modèle peut se tromper de deux façons. Il peut bloquer un client honnête : c'est un faux positif, avec à la clé un paiement refusé et souvent une réclamation. Il peut aussi laisser passer une fraude : c'est un faux négatif, et la perte retombe sur la banque ou sur son client.

Mon rôle était celui d'une ingénieure en assurance qualité. Il ne s'agissait pas d'obtenir le meilleur score possible, mais de prouver, par des tests reproductibles, dans quelles conditions le système peut être jugé acceptable. J'ai donc passé autant de temps à chercher ce qui pouvait mal tourner qu'à entraîner des modèles.

Les données viennent du jeu public Credit Card Fraud Detection : 284 807 transactions de porteurs européens sur deux jours de septembre 2013, dont 492 fraudes, soit 0,17 %. En dehors du temps (`Time`) et du montant (`Amount`), les 28 variables sont des composantes anonymisées (`V1` à `V28`) issues d'une analyse en composantes principales.

Dans le périmètre	Hors périmètre
Contrôle du contrat de données, dédoublonnage	Collecte des données en amont, exactitude des labels Kaggle
Prétraitement, entraînement, choix du seuil	Modèles lourds (boosting, réseaux de neurones)
Sauvegarde, rechargement, prédiction, service de scoring	API HTTP réelle, authentification, infrastructure
Intégration continue avec GitHub Actions	Déploiement en production
Surveillance simulée (dérive, taux d'alertes)	Prometheus et Grafana
Analyse des écarts entre segments de transactions	Audit d'équité sur des groupes de clients (aucune donnée démographique)

2. Plan de test et matrice de traçabilité

J'ai écrit le plan de test avant la moindre expérience, pour ne pas risquer d'ajuster mes critères aux résultats. Il définit dix exigences : cinq fonctionnelles (rejet des données invalides, probabilité valide, seuil fixé sur la validation, rechargement fidèle, alerte sur les montants hors norme) et cinq non fonctionnelles (reproductibilité, performance prédictive,

latence, sécurité, intégration continue). La version complète figure dans `docs/plan_de_test.md`.

La matrice reprend les sept lignes du sujet et en ajoute dix-sept. Chaque ligne relie une exigence au risque qu'elle couvre ; toutes sont aujourd'hui vérifiées par au moins un test automatisé et toutes sont conformes.

ID	Exigence	Type	Risque maîtrisé
T01	Colonnes nécessaires présentes	Données	Prédiction sur un schéma incomplet
T02	Types conformes au contrat	Unitaire	Texte ou booléen converti en silence
T03	Aucune valeur non finie après prétraitement	Unitaire	Valeur manquante propagée jusqu'à la décision
T04	Probabilité comprise entre 0 et 1	Fonctionnel ML	Score inexploitable en aval
T05	Évaluation sur des données indépendantes	ML	Performance surestimée
T06	Résultats reproductibles	Non-régression	Résultats impossibles à auditer
T07	Tests exécutés sur GitHub	CI	Régression non détectée
T08	Rejet des valeurs manquantes, infinies, de la cible invalide	Données	Lot corrompu en silence
T09	Pas de fuite (scaler, doublons, partition)	Intégration	Modèle qui « reconnaît » le test
T10	Proportion de fraudes préservée	Intégration	Jeu de test sans fraude ou biaisé
T11	Entrées de l'API validées strictement	Sécurité	Champ injecté, mauvais typage
T12	Seuil fixé selon une règle écrite d'avance	ML	Seuil choisi après coup sur le test
T13	Rechargement identique, fichier altéré refusé	Intégration	Modèle déployé différent du modèle qualifié
T14	Aucune donnée de transaction dans les journaux	Sécurité	Fuite de données bancaires
T15	Montants extrêmes traités et signalés	Robustesse	Extrapolation silencieuse
T16	Effet limité d'une petite perturbation	Robustesse	Décisions instables
T17	Basculements limités aux cas proches du seuil	Robustesse	Décisions erratiques
T18	Dérive de distribution détectée	Surveillance	Dégradation invisible
T19	Pas de régression au-delà des tolérances	Non-régression	Nouvelle version moins bonne
T20	Aucun secret dans le dépôt	Sécurité	Comptes compromis
T21	Données brutes et modèles non versionnés	Conformité	Diffusion non autorisée
T22	Latence et taille dans les limites	Performance	Délai d'autorisation dépassé
T23	Métriques adaptées à la classe rare	Unitaire ML	Accuracy trompeuse
T24	Erreurs de chargement explicites	Intégration	Panne opaque en exploitation

Les critères d'acceptation du modèle ont été fixés au même moment, à partir des besoins de la banque et non des résultats.

Critère	Seuil	Pourquoi ce seuil
PR-AUC	au moins 0,70	Un modèle au hasard obtiendrait 0,0017 sur ces données.

Critère	Seuil	Pourquoi ce seuil
Rappel	au moins 0,75	Une fraude manquée coûte bien plus cher qu'une vérification.
Précision	au moins 0,50	Au plus une fausse alerte par fraude détectée, pour ne pas noyer les analystes.
Taux de faux positifs	au plus 0,1 %	Au plus un client honnête bloqué sur mille.
Latence P95	au plus 50 ms	Le scoring n'est qu'une étape d'une autorisation de quelques centaines de millisecondes.
Taille du modèle	au plus 10 Mo	Chargement rapide, déploiement simple.

3. Architecture du pipeline et choix méthodologiques

Le code est découpé en modules dans `app/` (validation et prétraitement, entraînement, prédiction, évaluation, robustesse, surveillance), avec un seul fichier de configuration qui regroupe le contrat de données, la graine aléatoire, les critères et les tolérances. Les étapes s'enchaînent toujours dans le même ordre :

1. les données brutes sont contrôlées, puis débarrassées des doublons exacts ;
2. elles sont partagées en entraînement, validation et test ;
3. un `Pipeline` scikit-learn (transformations et modèle) est ajusté sur l'entraînement seul ;
4. la validation sert à choisir les hyperparamètres, le modèle et le seuil de décision ;
5. le modèle retenu est sauvegardé avec une empreinte SHA-256 ;
6. le test n'intervient qu'une fois, pour l'évaluation finale du modèle rechargé.

Voici les choix qui m'ont paru les plus importants, et leur raison.

Choix	Raison
Supprimer les doublons avant de partager les données	J'ai trouvé 1 081 lignes strictement identiques, dont 19 fraudes. Laissées en place, certaines se seraient retrouvées à la fois dans l'entraînement et dans le test : le modèle aurait « reconnu » des fraudes au lieu de les détecter.
Partage 60 / 20 / 20 stratifié, graine 42	Chaque jeu garde la même proportion de fraudes (0,167 %), soit environ 95 fraudes par jeu, et le découpage se rejoue à l'identique.
Tout regrouper dans un Pipeline	La standardisation est ajustée sur l'entraînement seul puis appliquée telle quelle : impossible de l'ajuster par erreur sur le test.
Montant passé au logarithme (log1p)	Sa distribution est très déséquilibrée : médiane de 22 €, maximum de 25 691 €.
Deux modèles légers départagés sur la validation	Régression logistique et arbre de décision, chacun réglé sur une petite grille. La PR-AUC sert d'arbitre car elle reste parlante quand la classe positive est rare. L'arbre l'emporte : 0,833 contre 0,798.
Seuil fixé par une règle écrite d'avance	Parmi les seuils qui donnent au moins 80 % de rappel et au plus 0,1 % de faux positifs en validation, je garde celui qui maximise la précision. Pour l'arbre, c'est 0,30.
Rejeter plutôt que deviner	Une valeur manquante, infinie ou mal typée entraîne un refus explicite. Remplacer un trou par une moyenne reviendrait à inventer une transaction.
Sauvegarde avec empreinte	L'empreinte est vérifiée avant tout chargement : un fichier modifié ou abîmé est refusé.

Un test « piège » vérifie le point le plus délicat. Il remplace le jeu de test par un objet qui provoque une erreur au premier accès, puis lance tout le réglage. Le test passe, ce qui prouve que le réglage ne lit jamais le jeu de test.

4. Résultats des tests unitaires et d'intégration

Les 136 tests passent, aucun n'échoue, et ils couvrent 97 % du code (la CI exige au moins 85 %). L'outil Ruff ne relève aucun problème de style ni de sécurité. Les tests tournent automatiquement à chaque modification, sous Python 3.12 et 3.13 ; les captures d'exécution figurent en annexe.

Fichier	Tests	Ce qui est vérifié
`test_preprocessing.py`	40	colonnes requises, absentes ou dupliquées, types, valeurs manquantes et infinies, jeu vide, cible hors {0, 1}, une seule observation, dimensions, montants extrêmes, chargement
`test_prediction.py`	38	contrat de sortie, seuil, entraînement, métriques comparées à un calcul fait à la main, règle de seuil, service de scoring, journaux
`test_robustness.py`	22	les six scénarios du sujet, sensibilité au bruit, propriétés générées avec Hypothesis, dérive
`test_integration.py`	15	scénario de bout en bout, partition, fuite du scaler, test jamais lu, rechargement, fichiers corrompus ou modifiés
`test_regression.py`	9	reproductibilité, règle de décision, non-régression sur trois niveaux de données
`test_security.py`	8	secrets, fichiers d'identifiants, .gitignore, fichiers volumineux
`test_performance.py`	4	taille du modèle, latence P95, débit

J'ai veillé à ce que chaque test vérifie un comportement précis, et pas seulement l'absence d'erreur. Quand une colonne manque, par exemple, le test exige que le message d'erreur la nomme. Quand on recharge le modèle, ses prédictions doivent coïncider à 10^{-12} près avec celles d'avant la sauvegarde.

Les tests unitaires tournent sur des données synthétiques qui reprennent le schéma Kaggle. C'est un choix volontaire : la CI n'a pas à télécharger un fichier de 150 Mo pour vérifier le code. Les contrôles sur les vraies données passent par un échantillon de 5 000 lignes versionné dans le dépôt, et par un test lent sur le jeu complet, lancé à la main.

5. Métriques de performance et matrices de confusion

Sur le jeu de test (95 fraudes, 56 651 transactions normales), l'arbre et la régression logistique font presque jeu égal. L'arbre a été retenu sur la validation, avant que le test ne soit ouvert.

Métrique (test)	Régression logistique	Arbre de décision (retenu)	Intervalle à 95 % (arbre)
PR-AUC	0,698	0,724	0,63 à 0,81
ROC-AUC	0,949	0,939	
Précision	0,843	0,852	0,77 à 0,92
Rappel	0,737	0,726	0,64 à 0,81
F1	0,787	0,784	

Métrique (test)	Régression logistique	Arbre de décision (retenu)	Intervalle à 95 % (arbre)
Taux de faux positifs	0,023 %	0,021 %	0,011 % à 0,034 %
Taux de faux négatifs	26,3 %	27,4 %	
Accuracy	99,933 %	99,933 %	

Les intervalles ont été calculés par bootstrap, sur 1 000 tirages.

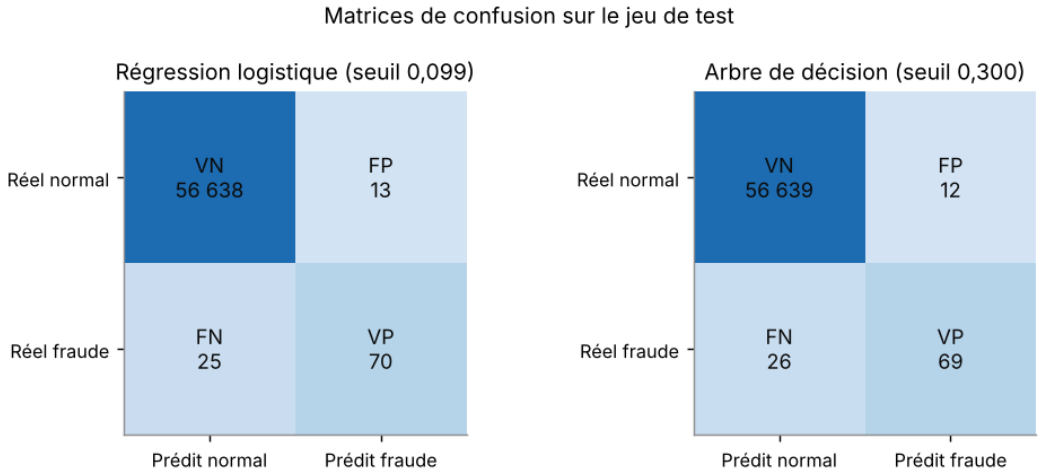


Figure 1. Matrices de confusion sur le jeu de test (VN, FP, FN, VP).

Concrètement, sur près de 57 000 paiements, l'arbre bloque 12 clients honnêtes et laisse passer 26 fraudes. En montant, il arrête 10 080 € sur les 14 766 € de fraude du test, soit 68 %.

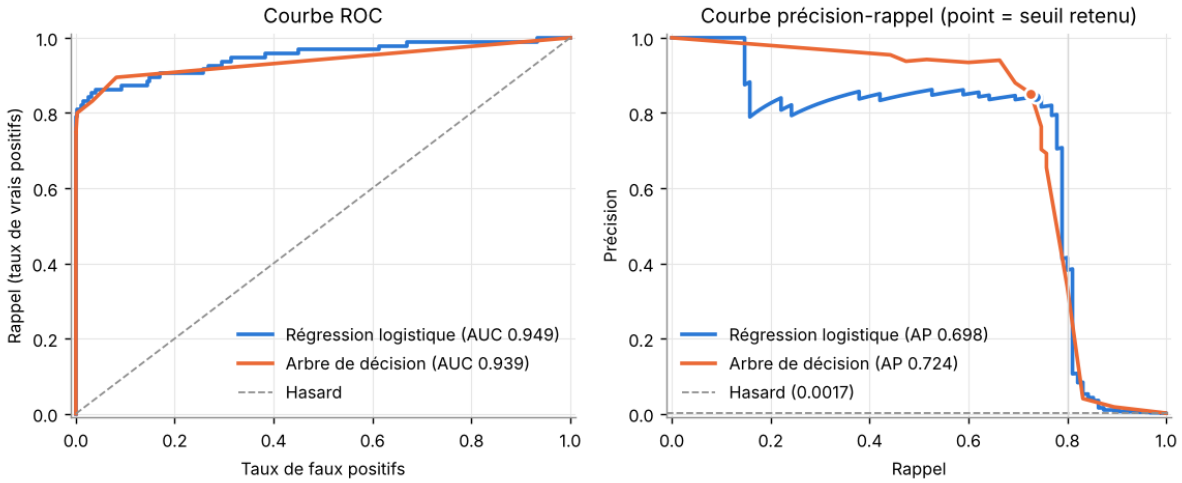


Figure 2. Courbes ROC et précision-rappel sur le jeu de test (le point marque le seuil retenu).

La courbe ROC flatte les deux modèles avec des aires proches de 0,95. La courbe précision-rappel est plus honnête ici : les deux modèles gardent une précision supérieure à 0,8 jusqu'à environ 75 % de rappel, puis s'effondrent. Au-delà, chaque fraude supplémentaire coûterait des dizaines, puis des centaines de fausses alertes.

L'accuracy ne m'a servi à rien pour décider. Un « modèle » qui déclarerait toutes les transactions normales atteindrait déjà 99,83 %, sans détecter une seule fraude. Toute la différence entre cet outil inutile et le mien tient dans un dixième de point.

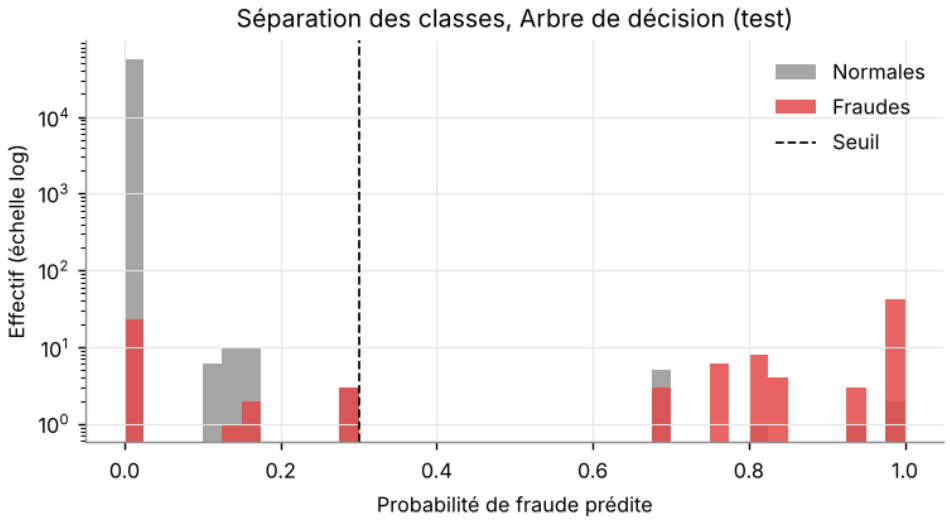


Figure 3. Scores attribués par l'arbre aux transactions normales et frauduleuses (échelle logarithmique).

L'analyse des erreurs m'a appris deux choses. D'abord, les fraudes manquées ne sont pas « juste sous le seuil » : la moitié reçoit un score inférieur à 0,0015. Baisser le seuil ne les rattraperait donc pas, il faudrait d'autres informations sur la transaction. Ensuite, ce sont surtout de petits montants (médiane de 3,78 €), souvent utilisés par les fraudeurs pour tester une carte volée avant un achat plus important.

Enfin, la validation s'est montrée optimiste. Le rappel de l'arbre y était de 0,851 ; il tombe à 0,726 sur le test avec le même seuil. Comme le modèle et le seuil ont été choisis pour briller sur la validation, celle-ci surestime mécaniquement les performances. Avec moins de 100 fraudes par jeu, quelques cas difficiles de plus ou de moins suffisent à creuser l'écart.

6. Tests de robustesse et de non-régression

Données imparfaites

Pour chaque scénario du sujet, j'ai d'abord écrit le comportement attendu, puis je l'ai confronté au comportement observé sur le vrai jeu de test. Les dix-sept cas se sont comportés comme prévu.

Scénario	Attendu	Observé
Valeurs manquantes dans une ou trois variables	rejet	rejet, variable et nombre de valeurs nommés
Valeurs infinies, valeur non numérique (« 12,50 »)	rejet	rejet
Champ manquant, champ en trop, booléen dans une requête	rejet	rejet, champ nommé
Montant égal à 2, 10 puis 1 000 fois le maximum vu à l'entraînement	traitement et alerte	traité, toutes les lignes alertées
Décalage de 0,5 écart type sur V10, V12 et V14	alerte de dérive	alerte sur exactement ces trois variables
Colonne absente, renommée ou dupliquée, lot vide	rejet	rejet
Colonnes dans un autre ordre	traitement	réordonnées, prédictions identiques

Pourquoi rejeter plutôt que corriger ? Une transaction à laquelle il manque une variable n'est pas une transaction « presque normale » : elle signale un problème en amont. Lui inventer une valeur reviendrait à fabriquer un score. Le rejet laisse le circuit de repli de la banque prendre le relais.

Un de ces scénarios m'a mise sur la piste d'une anomalie. Multiplier tous les montants par 1 000 ne change strictement aucune décision. Ce n'est pas de la robustesse : l'arbre n'utilise tout simplement pas la variable `Amount`. Une transaction de 19 millions d'euros est jugée comme une transaction de 10 €.

Sensibilité aux petites perturbations

Je n'ai pas appliqué le même bruit à toutes les variables. Les composantes `V1` à `V28` ont reçu un bruit proportionnel à leur propre écart type, le montant une variation de quelques pour cent (arrondi, conversion, frais), et le temps un décalage d'une minute ou d'une heure (horloge désynchronisée).

Perturbation	Décisions changées	dont fraudes	Effet sur le rappel	Effet sur la précision
Bruit de 1 % sur V1 à V28	0	0	0	0
Bruit de 5 %	3	1	-0,011	-0,023
Bruit de 10 %	4	1	-0,011	-0,033
Bruit de 25 %	5	2	0	-0,030
Montant modifié de 1 à 20 %	0	0	0	0
Temps décalé d'une minute	0	0	0	0
Temps décalé d'une heure	1	0	0	-0,010

Sur 56 746 transactions, le modèle est donc très stable en moyenne. Mais quand une décision change, elle change d'un coup, avec un score qui peut passer de 0 à presque 1 : c'est la marque d'un arbre, dont les 19 feuilles découpent l'espace en zones aux frontières nettes. Plus inquiétant, un simple décalage d'une heure sur `Time` suffit à faire basculer une transaction. Le modèle a donc appris quelque chose de la position dans le jeu Kaggle, une information qui n'existera pas en production.

Une dérive réelle entre les deux jours

En comparant le premier jour au second, 11 variables sur 29 dépassent le seuil d'alerte de 0,20 sur l'indice de stabilité (PSI). La médiane de V1 passe par exemple de -0,28 à 0,11. Quand j'entraîne le modèle sur les transactions les plus anciennes et que je le teste sur les plus récentes, sa précision tombe de 0,85 à 0,74. La partition aléatoire, qui mélange les deux jours, donne donc une image plus favorable que la réalité.

Non-régression

La référence a été produite une seule fois et figée. Elle enregistre la version du code, les paramètres, les versions des bibliothèques, la graine, le seuil, l'empreinte du jeu de données et les métriques de validation. Elle n'est jamais recalculée automatiquement, sinon une régression passerait inaperçue. Une nouvelle version est rejetée si une métrique se dégrade au-delà de sa tolérance (0,02 pour la PR-AUC, 0,01 pour la ROC-AUC, 0,05 pour le rappel et la précision, 0,0005 pour le taux de faux positifs).

Version	PR-AUC	Rappel	Précision	Décision
v1.0 référence (arbre, profondeur 6)	0,833	0,851	0,870	référence
v1.1 arbre de profondeur 8	0,832	0,851	0,870	accepter
v1.2 arbre avec classes pondérées	0,466	0,777	0,579	rejeter
v1.3 régression logistique	0,798	0,809	0,776	rejeter
v1.4 arbre de profondeur 1 (régression volontaire)	0,556	0,713	0,779	rejeter

La version 1.4 sert de démonstration. Lancée sur GitHub avec l'option d'injection, elle fait échouer les deux tests de non-régression, et le workflow passe au rouge (captures en annexe). Un premier essai m'a d'ailleurs surprise : une régularisation beaucoup plus forte de la régression logistique ne déclenchait aucun échec. Elle réduit la taille des coefficients sans changer leur direction, donc l'ordre des transactions reste le même et le seuil se réajuste. Les métriques de classement n'y voient rien, ce qui montre leurs limites.

7. Latence et consommation de ressources

Les mesures ont été faites sur une machine Linux à 2 processeurs et 8,4 Go de mémoire, avec 50 appels d'échauffement puis 2 000 appels mesurés. Le débit est la médiane de 5 répétitions. L'appel de « service » comprend la validation stricte de la requête, mais pas le réseau : c'est une borne basse de ce que verrait un vrai système d'autorisation.

Mesure	Valeur	Critère
Taille du modèle sauvegardé	11 Ko	au plus 10 Mo, conforme
Temps de chargement	1,8 ms	
Prédiction seule, médiane / P95 / P99	3,2 / 4,8 / 5,3 ms	
Appel de service, médiane / P95 / P99	10,3 / 13,4 / 15,9 ms	P95 au plus 50 ms, conforme
Débit, transactions une par une	288 par seconde	
Débit, lot de 56 746 transactions	4,7 millions par seconde	
Mémoire ajoutée par le modèle	0,25 Mo	

Le modèle lui-même ne coûte presque rien : un arbre de 19 feuilles fait au plus six comparaisons par transaction. Ce qui coûte, c'est tout ce qui l'entoure, la construction des tableaux de données et surtout la validation, qui triple le temps de réponse. Sur 2 000 prédictions, une seule a pris 101 ms, un pic ponctuel sans doute dû au système. Il justifie de suivre le P99 et le maximum en production, et pas seulement la médiane.

8. Anomalies, gravité et statut

N°	Anomalie	Gravité	Statut
A1	1 081 transactions en double (dont 19 fraudes) pouvant se retrouver dans l'entraînement et le test	majeure	corrigée : dédoublonnage avant partition, couvert par un test
A2	Rappel de 0,726 en test, sous le critère de 0,75	majeure	ouverte : écart dans l'intervalle de confiance, à confirmer sur de vraies données

N°	Anomalie	Gravité	Statut
A3	Rappel de 0,851 en validation contre 0,726 en test : validation optimiste	majeure	ouverte : prévoir une validation croisée et une validation dans le temps
A4	Le modèle utilise Time, un compteur relatif au début du jeu Kaggle ; un décalage d'une heure change une décision	bloquante	ouverte : retirer Time ou le remplacer par l'heure de la journée, puis requalifier
A5	Le modèle ignore le montant ; son rappel n'est que de 0,65 sur les fraudes de plus de 100 €	majeure	ouverte : tester des variables issues du montant
A6	Dérive marquée entre les deux jours ; précision de 0,74 hors temps	majeure	ouverte : surveillance par tranche horaire, réentraînement planifié
A7	Le taux d'alertes et la distribution des scores ne voient pas une forte dérive de V14	moyenne	traitée : la surveillance des entrées est prévue
A8	Les alertes sur le taux de signalement se déclenchent chaque nuit (rythme, pas panne)	mineure	ouverte : références par tranche horaire
A9	Une forte régularisation échappe aux tests fondés sur les métriques de classement	mineure	close : documentée, contrôle de calibration recommandé
A10	Pic isolé de 101 ms sur une prédiction	mineure	ouverte : suivre P99 et maximum

9. Limites de l'évaluation et risques résiduels

Limites des données. Les transactions datent de 2013, ne couvrent que deux jours et viennent d'une seule source. Les fraudes actuelles (paiement mobile, hameçonnage, prise de contrôle de compte) n'y figurent probablement pas. Les variables anonymisées empêchent toute interprétation métier. Surtout, le jeu ne contient aucune information sur les clients (âge, sexe, région), si bien qu'il est impossible de conclure sur l'équité du modèle envers des groupes de personnes. Je n'ai pas cherché à deviner de tels groupes à partir des variables anonymisées : ce serait inventer des données.

Limites de l'évaluation. Avec 95 fraudes en test, le rappel n'est connu qu'à environ 9 points près. Les perturbations testées sont synthétiques et ne reproduisent pas un fraudeur qui adapte sa stratégie. La latence mesurée n'inclut ni le réseau ni une vraie API.

Risques résiduels si le modèle était déployé tel quel. Environ une fraude sur quatre passerait, notamment les tests de carte à petit montant et les fraudes de plus de 100 €. La dépendance à `Time` rendrait certaines décisions imprévisibles. Sans surveillance des variables d'entrée, une dérive pourrait dégrader le modèle pendant des semaines avant que les fraudes confirmées ne la révèlent.

10. Décision

Il faut distinguer deux questions. La première est de savoir si le système réussit les tests techniques. La réponse est oui : le contrat de données est respecté, il n'y a pas de fuite, les résultats sont reproductibles, les entrées invalides sont rejetées, les temps de réponse sont très en dessous des limites, le dépôt est sain et l'intégration continue fonctionne, échec volontaire compris.

La seconde est de savoir si le modèle peut être autorisé à agir sur de vrais paiements. La réponse est non, pas en l'état. Cette autorisation ne dépend d'ailleurs pas seulement de mes tests : elle demande aussi une validation métier par l'équipe de lutte contre la fraude,

une validation opérationnelle par les équipes d'exploitation, une validation de sécurité et une validation réglementaire, un tel système relevant vraisemblablement des usages à haut risque du règlement européen sur l'IA.

Je recommande donc un **pilote en mode observation** : le modèle score les transactions réelles en parallèle du dispositif actuel, sans jamais bloquer un paiement, et ses alertes sont comparées après coup aux décisions des analystes. Ce pilote est possible aux conditions suivantes :

1. corriger l'anomalie A4, en retirant ou en transformant la variable Time, puis rejouer toute la qualification ;
2. réentraîner sur des données récentes de la banque, couvrant plusieurs semaines, et évaluer sur une période postérieure à l'entraînement ;
3. faire valider les critères d'acceptation, en particulier le rappel minimal et le coût d'une fausse alerte, par le métier ;
4. activer la surveillance des variables d'entrée et du taux d'alertes, avec des références par tranche horaire ;
5. garder les règles actuelles de la banque comme seul mécanisme de blocage pendant toute la durée du pilote.

Avant une vraie mise en production, il faudra en plus des preuves que ce projet ne pouvait pas apporter : des performances mesurées sur plusieurs milliers de fraudes récentes, pour des intervalles de confiance de 2 à 3 points ; un coût des erreurs chiffré avec les équipes ; un audit d'équité sur des attributs clients autorisés, mené dans le respect du RGPD ; des tests de charge sur l'API réelle ; et une surveillance éprouvée pendant le pilote, avec une procédure de retour arrière testée.

En résumé, ce travail montre que le système est testable, et testé. Il ne montre pas encore que le modèle est bon pour la banque : c'est précisément le rôle du pilote.

Question de synthèse

Dans quelles conditions le modèle peut-il être considéré comme suffisamment fiable pour être testé dans un environnement pilote, et quelles preuves supplémentaires seraient nécessaires avant une mise en production réelle ?

Les conditions d'un pilote

Pour moi, la première condition est que le pilote ne puisse nuire à personne. Le modèle doit tourner en parallèle du dispositif actuel, scorer les transactions sans jamais les bloquer, et ses alertes doivent être comparées après coup aux décisions des analystes. Dans ce cadre, mes résultats suffisent pour démarrer.

Le pipeline est fiable au sens technique. Il rejette ce qu'il ne comprend pas (17 scénarios de données imparfaites conformes), il donne toujours la même réponse à la même question (résultats identiques à graine égale, prédictions identiques après rechargement), il ne laisse fuir ni données ni secrets et il répond en 13,4 ms au 95e percentile, très loin de la limite de 50 ms. Côté modèle, les résultats obtenus sur des données qu'il n'avait jamais vues montrent qu'il signale très peu de clients honnêtes (0,021 %, contre 0,1 % tolérés) et qu'il se trompe rarement quand il signale (précision de 0,852). Ce profil convient bien à un pilote : une fausse alerte y coûte une vérification par un analyste, rien de plus.

Deux conditions doivent cependant être remplies avant le lancement. La première est technique : la dépendance à la variable Time doit disparaître (anomalie A4), sinon certaines décisions dépendraient d'un compteur qui n'existe pas en production. La seconde est métier : les critères d'acceptation doivent être validés par l'équipe de lutte contre la fraude. J'ai fixé le rappel minimal à 0,75 par raisonnement, et c'est précisément le critère que le modèle manque, avec 0,726. L'écart reste dans l'intervalle de confiance (0,64 à 0,81), ce qui ne permet ni de valider ni de rejeter le modèle sur ce seul point. Seul le métier peut dire si laisser passer une fraude sur quatre est acceptable en échange de très peu de fausses alertes.

Les preuves à réunir avant une mise en production

Plusieurs preuves manquent encore, et ce projet ne pouvait pas les apporter avec ce jeu de données.

- **Des performances mesurées sur les transactions de la banque.** Les données de 2013 couvrent deux jours seulement. Mes propres essais montrent que la précision tombe de 0,85 à 0,74 dès que l'on teste sur une période postérieure à l'entraînement : il faut une évaluation sur plusieurs mois de données récentes, toujours sur une période plus tardive que l'entraînement.
- **Assez de fraudes pour des résultats précis.** Avec 95 fraudes, le rappel n'est connu qu'à environ 9 points près. Une décision de production demande une précision de 2 à 3 points, donc plusieurs milliers de fraudes confirmées.
- **Un coût des erreurs chiffré avec les équipes.** Montant moyen d'une fraude manquée, coût d'une vérification, taux de réclamation après un blocage : ces chiffres permettraient de choisir le seuil sur une base économique, et non sur mes hypothèses. L'enjeu est réel, car le modèle ignore le montant et ne détecte que 65 % des fraudes de plus de 100 €.
- **Un audit d'équité.** Le jeu ne contient aucune donnée sur les clients. Il faudra mesurer les écarts de faux positifs et de rappel selon des attributs autorisés (tranche d'âge, région, type de carte), dans le respect du RGPD.
- **Des tests sur l'API réelle.** Charge, authentification, temps de réponse réseau compris : rien de cela n'a pu être mesuré sur le prototype.
- **Une surveillance éprouvée pendant le pilote.** Les seuils d'alerte doivent être étalonnés sur l'historique, par tranche horaire, puisque la simulation a montré que le taux d'alertes et certaines variables varient naturellement la nuit. Une procédure de retour arrière doit aussi avoir été testée.
- **L'accord des fonctions risques, conformité et sécurité.** Un système qui peut refuser un paiement relève vraisemblablement des usages à haut risque du règlement européen sur l'IA.

En résumé, le modèle est assez fiable pour être observé, pas encore pour décider. Le pilote servira justement à réunir les preuves qui manquent.

Annexe. Preuves d'exécution sur GitHub Actions

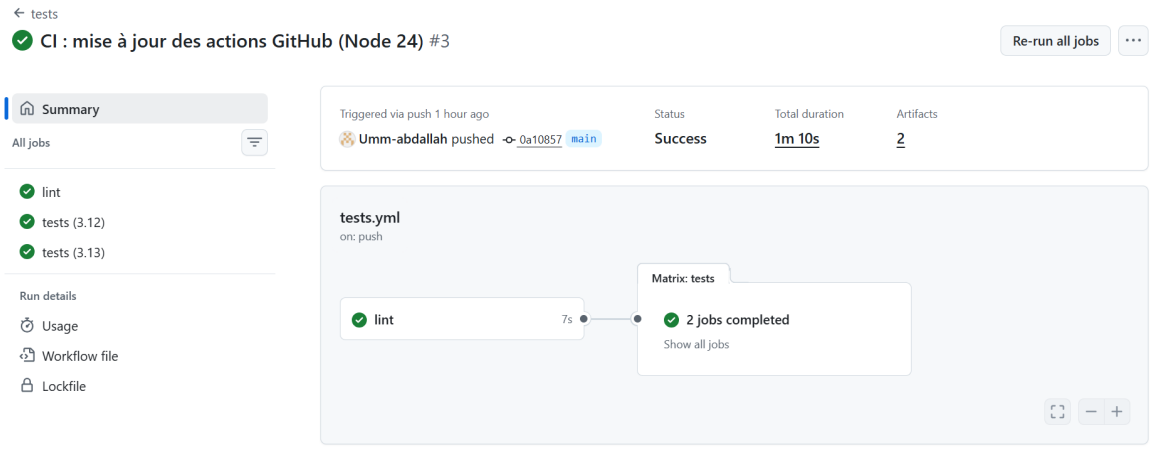


Figure 4. Exécution réussie du workflow tests (n°3) : lint et tests sous Python 3.12 et 3.13.

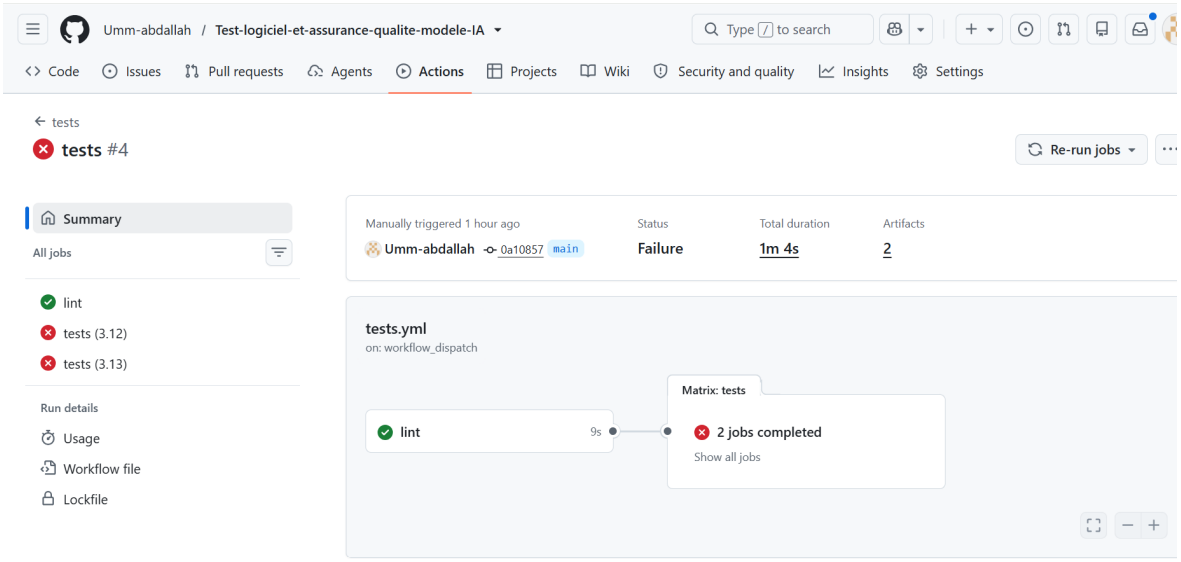


Figure 5. Exécution n°4 avec régression volontaire injectée : les deux jobs de tests échouent, le workflow est en échec.

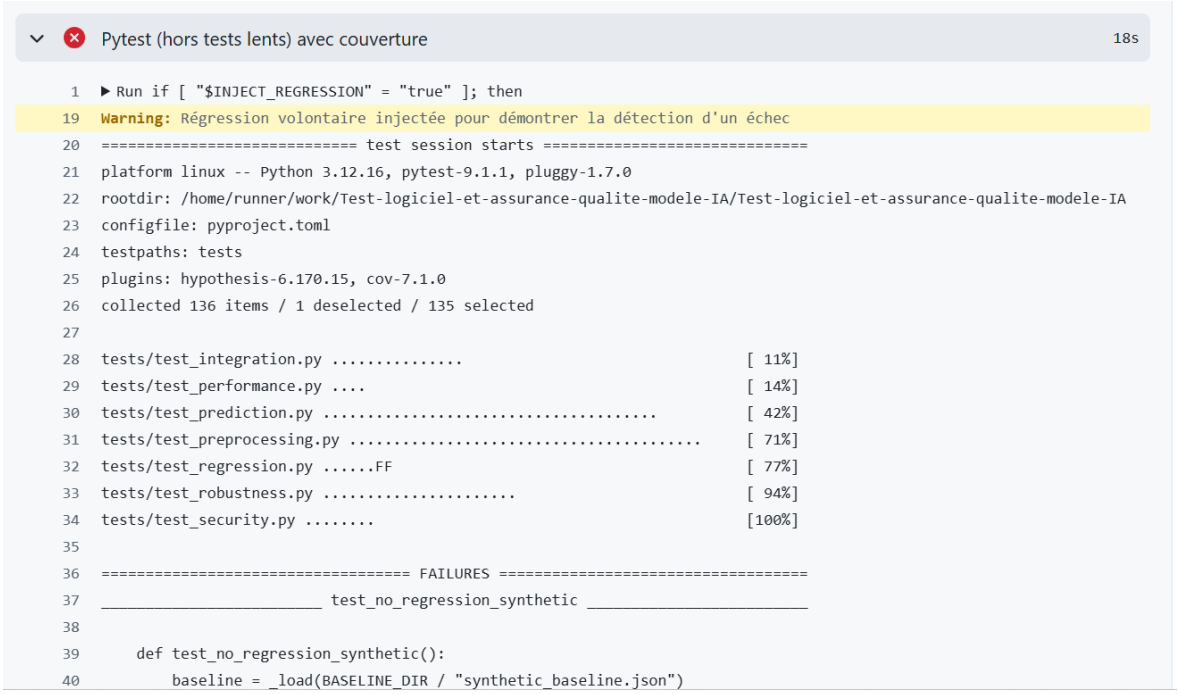


Figure 6. Détail de l'échec : avertissement d'injection et échec des deux tests de non-régression.

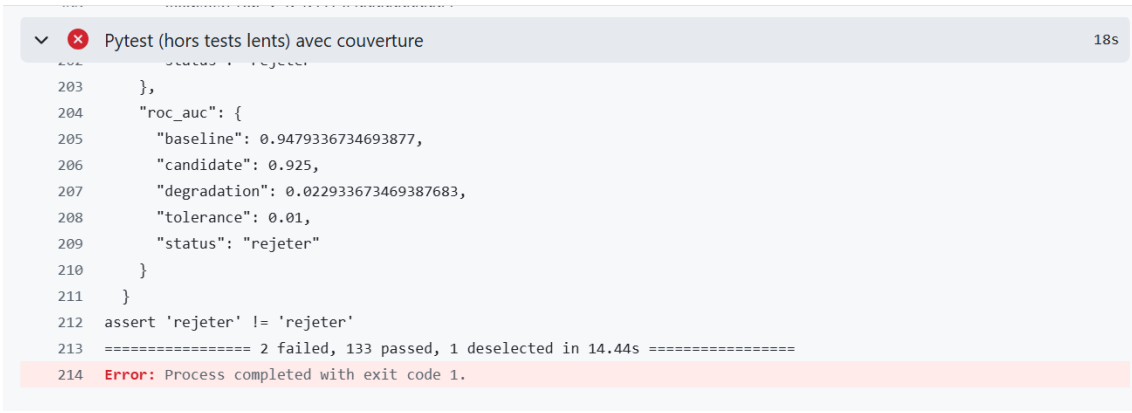


Figure 7. Fin du journal : dégradation au-delà de la tolérance, décision « rejeter », 2 tests échoués.

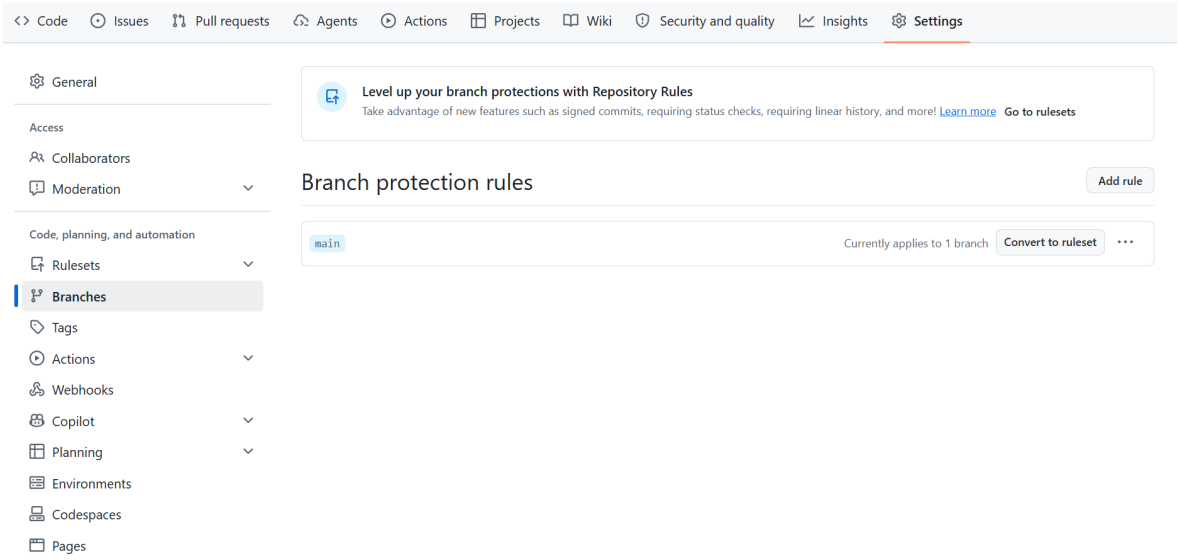


Figure 8. Règle de protection active sur la branche main.

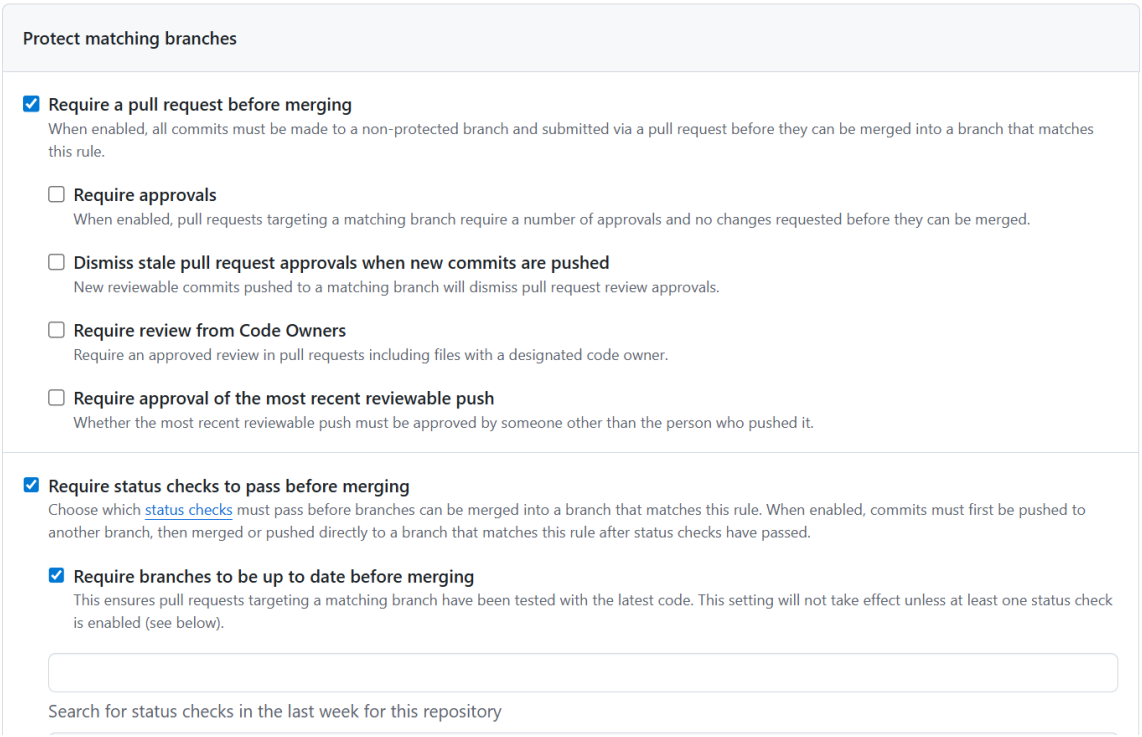


Figure 9. Pull request obligatoire avant toute fusion dans main.

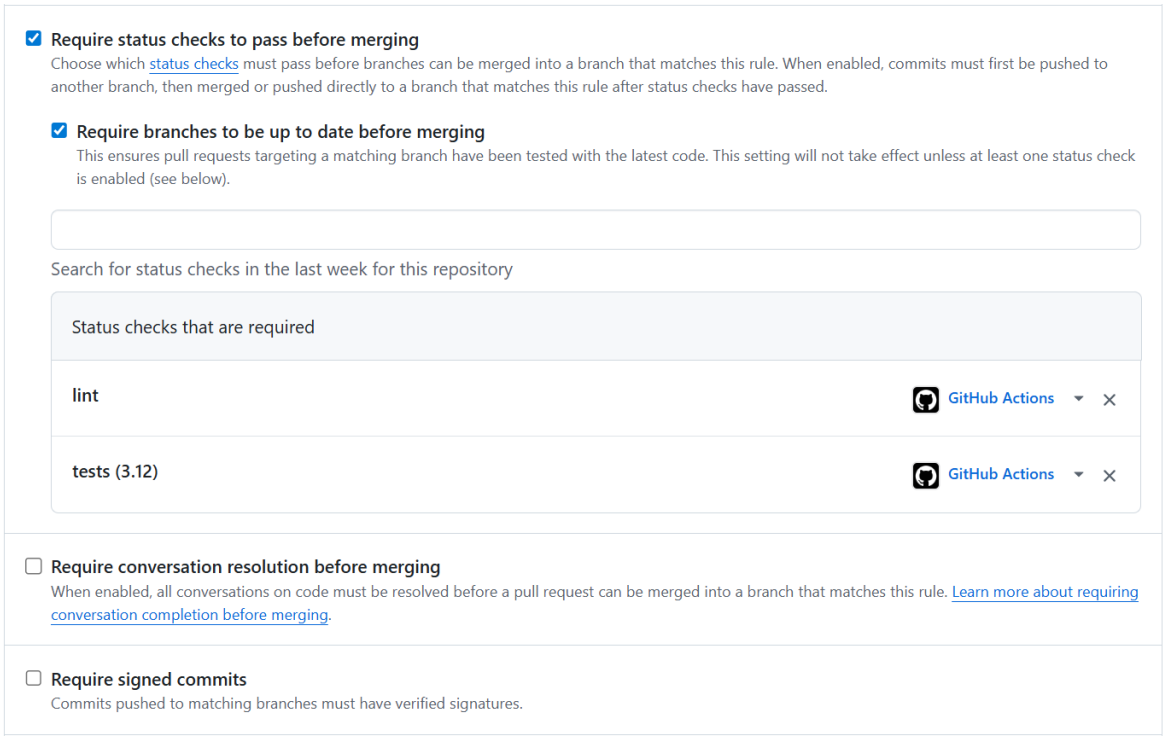


Figure 10. Vérifications obligatoires : lint et tests (3.12) doivent réussir.